



UNIVERSIDADE ESTADUAL PAULISTA "JÚLIO DE MESQUITA FILHO"

CÂMPUS DE PRESIDENTE PRUDENTE

FACULDADE DE CIÊNCIAS E TECNOLOGIA DEPARTAMENTO DA MATEMÁTICA
E COMPUTAÇÃO

BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

Luan Yudi Uemura Kishimoto

Previsão da Velocidade do Vento - Parte 2

Presidente Prudente

2025

1. Introdução

Com o avanço das mudanças climáticas e a crescente disponibilidade de dados ambientais, a aplicação de técnicas de Data Science tornou-se essencial para a análise e previsão de variáveis meteorológicas. Neste contexto, a velocidade do vento se destaca como uma variável de grande relevância, tanto por seu impacto em eventos climáticos extremos quanto por seu papel em setores como energia, aviação e agricultura.

Este estudo está estruturado em duas partes principais, com foco na aplicação de um pipeline de ciência de dados para prever a velocidade do vento a partir de dados meteorológicos históricos. Inicialmente, são realizadas análises exploratórias e estatísticas descritivas para compreender os padrões e a variabilidade presentes nos dados. Posteriormente, são criadas lag features — variáveis que representam defasagens temporais — com o objetivo de capturar dependências temporais que possam aprimorar a capacidade preditiva dos modelos.

A modelagem envolve a comparação entre diferentes algoritmos de regressão, abrangendo modelos lineares, como Ridge e Lasso, e não lineares, como Random Forest e XGBoost. Os modelos são avaliados em duas configurações distintas: uma utilizando as lag features para explorar informações históricas, e outra considerando apenas as variáveis originais, sem defasagens temporais. O ajuste automático dos hiperparâmetros é realizado por meio do Optuna, garantindo a otimização do desempenho. Por fim, para tornar os resultados mais interpretáveis, aplicam-se técnicas de Explainable AI (XAI), como LIME e SHAP, que permitem identificar quais variáveis e quais lags exercem maior influência nas previsões.

2. Implementação dos Modelos de Regressão

A previsão da velocidade do vento é uma tarefa de regressão, já que envolve a estimativa de um valor numérico a partir de variáveis meteorológicas explicativas. Modelos de regressão são especialmente adequados para esse tipo de problema, pois buscam identificar e quantificar a relação entre as variáveis independentes e a variável alvo.

Para obter um panorama completo das capacidades preditivas, foram selecionados diferentes modelos de regressão que abrangem desde abordagens lineares simples até métodos mais avançados e não lineares. Essa diversidade

permite explorar tanto relações lineares básicas quanto padrões complexos presentes nos dados, além de possibilitar a comparação entre interpretabilidade e desempenho preditivo:

- Regressão Linear: modelo mais básico de regressão, que assume uma relação linear entre as variáveis independentes e a variável alvo; sendo simples, rápida e de fácil interpretação.
- Regressão Ridge: extensão da regressão linear que incorpora penalização L2 que ajuda a controlar o overfitting(aprende padrões reais, mas também ruídos e variações aleatórias) e a multicolinearidade entre variáveis. Essa regularização torna o modelo mais estável e capaz de generalizar melhor em dados novos.
- Regressão Lasso: modelo regularizado que utiliza penalização L1(técnica de regularização), promovendo a seleção automática de variáveis ao forçar alguns coeficientes a zero, facilitando a interpretação e melhorando o desempenho ao focar nas variáveis mais relevantes.
- Random Forest: modelo de ensemble(combina modelos fracos ou individuais para formar um modelo forte e preciso) baseado em múltiplas árvores de decisão. É resistente a ruídos e geralmente apresenta bom desempenho em diversas tarefas de regressão.
- XGBoost: Algoritmo de boosting que constrói sequencialmente árvores de decisão, corrigindo erros de modelos anteriores. É altamente eficiente, com mecanismos de regularização para evitar overfitting, e é conhecido por entregar alta acurácia em problemas complexos e conjuntos de dados heterogêneos.

Para avaliar como os modelos de regressão estão de acordo com a tarefa de prever a velocidade do vento, foram usadas três métricas principais: MAE, MSE e R^2 . Cada métrica ajuda a avaliar os erros do modelo de um jeito diferente, dando uma visão mais completa do desempenho.

- Erro Absoluto Médio(MAE): mostra o quanto o modelo erra em média, sem se preocupar com a direção do erro. Por exemplo, se o modelo previu 5 knots e o valor real era 6 knots, o erro é 1. Como ele usa apenas o valor absoluto das

diferenças, é fácil de entender e útil para saber o tamanho típico dos erros em unidades reais (knots).

- Erro Quadrático Médio(MSE): também mede os erros, mas dá mais peso aos erros maiores. Isso acontece porque os erros são elevados ao quadrado, se o modelo cometer um erro muito grande em algum ponto, o MSE vai aumentar. Essa métrica é útil quando se deseja que o modelo evite grandes desvios, sendo muito usada durante o processo de ajuste automático dos modelos.
- Coeficiente de Determinação(R^2): mostra o quanto o modelo consegue explicar a variação dos dados. Variando de 0 a 1, os valores mais próximos de 1 indicam que o modelo está explicando bem os dados, em contrapartida, caso seja 0, significa que o modelo não está fazendo nada melhor do que estimar a média dos valores. É uma métrica boa para comparar diferentes modelos e ver qual deles entende melhor os padrões dos dados.

Essas três métricas foram usadas para avaliar todos os modelos testados no projeto, tanto com quanto sem as lag features. Assim, foi possível entender não só o quanto os modelos acertam, mas também como se comportam em relação a erros grandes e à explicação geral dos dados.

```
Treinando modelo random_forest...
random_forest - MAE: 2.9750 | RMSE: 3.8029 | R²: 0.4141

Treinando modelo xgboost...
xgboost - MAE: 3.0620 | RMSE: 3.9220 | R²: 0.3768

Treinando modelo ridge...
ridge - MAE: 2.9752 | RMSE: 3.8270 | R²: 0.4066

Treinando modelo linear_regression...
linear_regression - MAE: 2.9766 | RMSE: 3.8310 | R²: 0.4054

Treinando modelo lasso...
lasso - MAE: 3.0324 | RMSE: 3.8674 | R²: 0.3940
```

3. Hiperparametrização com Optuna

Durante o treinamento dos modelos citados anteriormente, existem determinados parâmetros que não são aprendidos automaticamente pelos dados, chamados de hiperparâmetros.

Cada tipo de modelo tem seus próprios hiperparâmetros. Random Forest, por exemplo, possui o número de árvores (`n_estimators`) e a profundidade máxima de cada árvore (`max_depth`); XGBoost, o `learning_rate`, `n_estimators` e `max_depth` como os principais; em modelos como Ridge e Lasso, o principal hiperparâmetro é o valor de `alpha`, que controla a força da regularização.

Dessa forma, escolher os melhores valores para esses hiperparâmetros pode fazer uma grande diferença na qualidade das previsões. No entanto, testar manualmente todas as combinações possíveis seria muito demorado — ainda mais quando se toma em conta o quanto o número de parâmetros cresce.

É aí que entra o Optuna, uma biblioteca que automatiza esse processo de busca, utilizando uma abordagem inteligente chamada "otimização bayesiana", o algoritmo tenta encontrar os melhores valores de forma eficiente, testando diferentes combinações e aprendendo com os resultados anteriores.

No projeto, o Optuna foi usado de modo a encontrar a combinação de hiperparâmetros que produz os melhores resultados de previsão da velocidade do vento para cada modelo tendo como funcionamento:

1. testar uma combinação de hiperparâmetros aleatórios;
2. treinar e avaliar com base em uma métrica de erro (como o MSE ou MAE);
3. escolher a próxima combinação de forma mais inteligente com base nos resultados;
4. repetir o processo por várias tentativas (chamadas de "trials");
5. No final, o Optuna retorna os melhores hiperparâmetros encontrados.

A imagem mostra como ocorre o processo no modelo Random Forest, onde se tem as tentativas (neste caso 29 trials) com cada valor de hiperparâmetro que mais se adequa, no final, ocorre a indicação de quais resultados são melhores para o determinado modelo, neste caso, `n_estimators`, `max_depth`, etc.

```
[I 2025-06-15 19:16:10,585] Trial 6 finished with value: 3.5330476564169735 and parameters: {'n_estimators': 275, 'max_depth': 30, 'min_samples_split': 9, 'min_samples
[I 2025-06-15 19:16:41,523] Trial 7 finished with value: 3.5362448927363497 and parameters: {'n_estimators': 478, 'max_depth': 17, 'min_samples_split': 8, 'min_samples
[I 2025-06-15 19:17:08,575] Trial 8 finished with value: 3.538845336189226 and parameters: {'n_estimators': 404, 'max_depth': 27, 'min_samples_split': 5, 'min_samples
[I 2025-06-15 19:17:24,292] Trial 9 finished with value: 3.533228169161408 and parameters: {'n_estimators': 250, 'max_depth': 23, 'min_samples_split': 4, 'min_samples
[I 2025-06-15 19:17:27,869] Trial 10 finished with value: 3.4967747123778365 and parameters: {'n_estimators': 115, 'max_depth': 5, 'min_samples_split': 10, 'min_sample
[I 2025-06-15 19:17:41,758] Trial 11 finished with value: 3.4942457258414343 and parameters: {'n_estimators': 350, 'max_depth': 6, 'min_samples_split': 2, 'min_samples
[I 2025-06-15 19:17:51,665] Trial 12 finished with value: 3.5063500547561937 and parameters: {'n_estimators': 175, 'max_depth': 9, 'min_samples_split': 4, 'min_samples
[I 2025-06-15 19:18:13,915] Trial 13 finished with value: 3.543514708444568 and parameters: {'n_estimators': 347, 'max_depth': 13, 'min_samples_split': 7, 'min_samples
[I 2025-06-15 19:18:20,783] Trial 14 finished with value: 3.5106949750473446 and parameters: {'n_estimators': 102, 'max_depth': 9, 'min_samples_split': 3, 'min_samples
[I 2025-06-15 19:18:35,695] Trial 15 finished with value: 3.536211052531904 and parameters: {'n_estimators': 228, 'max_depth': 14, 'min_samples_split': 10, 'min_sample
[I 2025-06-15 19:18:50,434] Trial 16 finished with value: 3.4953206608414016 and parameters: {'n_estimators': 433, 'max_depth': 5, 'min_samples_split': 6, 'min_samples
[I 2025-06-15 19:19:11,327] Trial 17 finished with value: 3.549276869257249 and parameters: {'n_estimators': 314, 'max_depth': 19, 'min_samples_split': 8, 'min_samples
[I 2025-06-15 19:19:32,285] Trial 18 finished with value: 3.5031736836667777 and parameters: {'n_estimators': 395, 'max_depth': 8, 'min_samples_split': 5, 'min_samples
[I 2025-06-15 19:19:40,873] Trial 19 finished with value: 3.5244958144903484 and parameters: {'n_estimators': 164, 'max_depth': 11, 'min_samples_split': 4, 'min_sample
[I 2025-06-15 19:20:02,121] Trial 20 finished with value: 3.532204051897893 and parameters: {'n_estimators': 308, 'max_depth': 22, 'min_samples_split': 3, 'min_samples
[I 2025-06-15 19:20:18,362] Trial 21 finished with value: 3.4971325825556194 and parameters: {'n_estimators': 345, 'max_depth': 7, 'min_samples_split': 2, 'min_samples
[I 2025-06-15 19:20:34,331] Trial 22 finished with value: 3.4940235511618263 and parameters: {'n_estimators': 388, 'max_depth': 6, 'min_samples_split': 2, 'min_samples
[I 2025-06-15 19:21:00,580] Trial 23 finished with value: 3.5254750183911754 and parameters: {'n_estimators': 405, 'max_depth': 11, 'min_samples_split': 3, 'min_sample
[I 2025-06-15 19:21:22,076] Trial 24 finished with value: 3.4981430805862024 and parameters: {'n_estimators': 479, 'max_depth': 7, 'min_samples_split': 2, 'min_samples
[I 2025-06-15 19:21:31,980] Trial 25 finished with value: 3.4954244247205573 and parameters: {'n_estimators': 248, 'max_depth': 5, 'min_samples_split': 3, 'min_samples
[I 2025-06-15 19:22:00,030] Trial 26 finished with value: 3.5470412289836695 and parameters: {'n_estimators': 378, 'max_depth': 16, 'min_samples_split': 7, 'min_sample
[I 2025-06-15 19:22:10,370] Trial 27 finished with value: 3.4996041068854477 and parameters: {'n_estimators': 205, 'max_depth': 8, 'min_samples_split': 5, 'min_samples
[I 2025-06-15 19:22:36,136] Trial 28 finished with value: 3.5175376394188285 and parameters: {'n_estimators': 431, 'max_depth': 11, 'min_samples_split': 9, 'min_sample
[I 2025-06-15 19:23:03,896] Trial 29 finished with value: 3.5735379327798698 and parameters: {'n_estimators': 315, 'max_depth': 15, 'min_samples_split': 2, 'min_sample
Melhores hiperparâmetros encontrados para random_forest: {'n_estimators': 219, 'max_depth': 7, 'min_samples_split': 9, 'min_samples_leaf': 5}

Treinando modelo random_forest...
[Fold 1] MAE: 3.7368 | RMSE: 4.7635 | R²: 0.1929
[Fold 2] MAE: 3.6427 | RMSE: 4.5095 | R²: 0.0284
[Fold 3] MAE: 3.2335 | RMSE: 4.1111 | R²: 0.1858
[Fold 4] MAE: 3.2777 | RMSE: 4.1313 | R²: 0.2286
[Fold 5] MAE: 3.5758 | RMSE: 4.5229 | R²: 0.1887
```

4. Separação de Dados: TimeSeriesSplit para Séries Temporais

No projeto, embora o TrainTestSplit — uma forma simples e comum de dividir os dados aleatoriamente entre treino e teste — tenha sido mencionado, a prioridade foi dada muito mais ao TimeSeriesSplit. Isso porque, ao lidar com séries temporais, como os dados meteorológicos, é fundamental respeitar a ordem cronológica das observações.

O TimeSeriesSplit é uma técnica de validação cruzada que divide os dados em múltiplos conjuntos chamados *folds*, sempre mantendo a sequência temporal intacta. Cada fold corresponde a uma divisão dos dados em blocos onde o modelo é treinado com dados até certo ponto no tempo e testado em dados posteriores. Essa estratégia evita que o modelo “veja o futuro” durante o treino, proporcionando uma avaliação mais realista e robusta do desempenho em situações reais.

Dessa forma, o uso do TimeSeriesSplit foi considerado mais adequado e foi a base principal para avaliação e otimização com Optuna, garantindo que o modelo fosse testado de forma coerente com a natureza sequencial dos dados de vento.

Na imagem, é possível notar a quantidade de fold's e diferentes valores para as métricas de MAE, MSE e R². A partir disso, ocorre o cálculo médio entre os 5 valores fornecidos de cada diferente modelo, mostrando então qual modelo melhor se sobressai em determinado ambiente.

```
Treinando modelo linear_regression...
[Fold 1] MAE: 3.8189 | RMSE: 4.8700 | R²: 0.1564
[Fold 2] MAE: 3.7560 | RMSE: 4.5891 | R²: -0.0062
[Fold 3] MAE: 3.4623 | RMSE: 4.3082 | R²: 0.1058
[Fold 4] MAE: 3.4062 | RMSE: 4.3082 | R²: 0.1612
[Fold 5] MAE: 3.6688 | RMSE: 4.6515 | R²: 0.1418

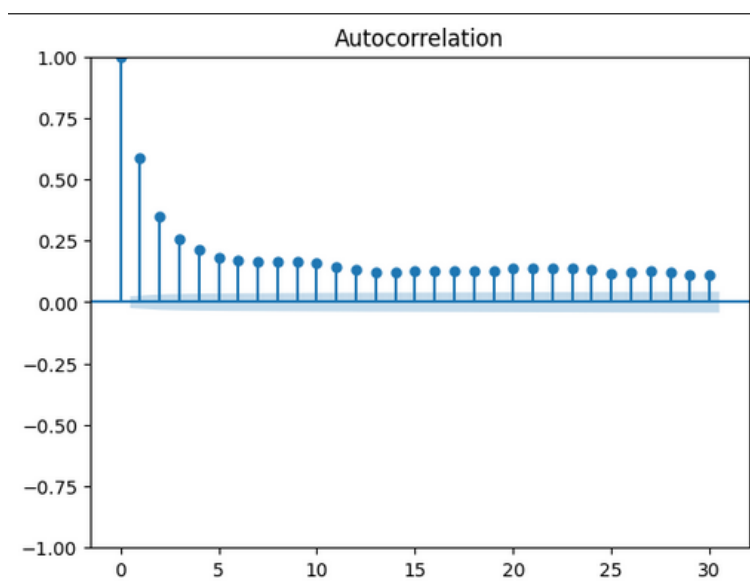
Resumo dos resultados médios por modelo:
```

	Modelo	MAE médio	RMSE médio	R² médio
1	xgboost	3.487216	4.388981	0.172000
0	random_forest	3.493311	4.407647	0.164854
2	ridge	3.612106	4.522293	0.119843
3	lasso	3.616692	4.521205	0.120199
4	linear_regression	3.622455	4.545390	0.111791

5. Inserção de Features no Dataset

Em problemas de séries temporais (como a previsão da velocidade do vento), é comum que o valor atual de uma variável dependa diretamente de seus valores passados. Para capturar essa dependência temporal, foram utilizadas lag features — ou seja, versões defasadas das variáveis de entrada.

Essas features representam os valores anteriores de uma variável em relação ao instante atual, como, por exemplo, a velocidade do vento 1 dia, ou até mesmo 5 dias antes. Ao incluir essas informações como entradas para o modelo, é possível fornecer um contexto histórico que ajuda a melhorar significativamente a capacidade preditiva. A seguir uma tabela que indica a correlação entre os valores atuais que estão fortemente relacionados aos valores imediatamente anteriores.



No desenvolvimento deste trabalho, as lag features se mostraram fundamentais para aumentar a qualidade das previsões. Modelos treinados com essas variáveis conseguiram capturar padrões de comportamento temporal mais complexos, resultando em melhores métricas de desempenho em comparação com modelos que não as utilizaram.

Resumo dos resultados médios por modelo:				
	Modelo	MAE médio	RMSE médio	R ² médio
1	xgboost	3.487216	4.388981	0.172000
0	random_forest	3.493311	4.407647	0.164854
2	ridge	3.612106	4.522293	0.119843
3	lasso	3.616692	4.521205	0.120199
4	linear_regression	3.622455	4.545390	0.111791

(Modelos sem lag features)

Resumo dos resultados médios por modelo:				
	Modelo	MAE médio	RMSE médio	R ² médio
0	random_forest	2.951069	3.751530	0.394715
1	xgboost	2.951615	3.754563	0.393712
3	lasso	2.981016	3.789457	0.381333
2	ridge	2.982183	3.791789	0.380627
4	linear_regression	2.984528	3.795000	0.379672

(Modelos com lag features)

Foram criadas também features voltadas para sazonalidade e interação entre as variáveis, que tem como objetivo principal melhorar a capacidade do modelo de capturar padrões e relações complexas entre os fatores climáticos.

Para representar a sazonalidade, ocorre a transformação dos meses e dias do ano em valores cíclicos, resolvendo o problema da descontinuidade entre meses como dezembro e janeiro, dessa forma, é possível entender que as estações são cíclicas e não lineares. Essa abordagem é mais eficiente do que simplesmente usar o número do mês como uma variável categórica, pois mantém a relação natural entre os períodos do ano.

Além disso, ocorre a criação de variáveis para ajudar o modelo a entender melhor como diferentes fatores climáticos se influenciam. A amplitude térmica, calculada pela diferença entre as temperaturas máxima e mínima do dia, pode indicar tempo instável ou estável. Já a interação entre chuva e temperatura máxima permite que o modelo diferencie entre tipos distintos de chuva, como as tempestades de verão (quentes) e as chuvas frontais de inverno (frias).

```
# Sazonalidade
df_work['month'] = df_work["DATE"].dt.month
df_work['day_of_year'] = df_work["DATE"].dt.dayofyear
df_work['month_sin'] = np.sin(2 * np.pi * df_work['month'] / 12)
df_work['month_cos'] = np.cos(2 * np.pi * df_work['month'] / 12)
df_work['day_sin'] = np.sin(2 * np.pi * df_work['day_of_year'] / 365)
df_work['day_cos'] = np.cos(2 * np.pi * df_work['day_of_year'] / 365)

# Interações
if 'T.MAX' in df_work.columns and 'T.MIN' in df_work.columns:
    df_work['temp_range'] = df_work['T.MAX'] - df_work['T.MIN']
if 'RAIN' in df_work.columns and 'T.MAX' in df_work.columns:
    df_work['rain_temp_interaction'] = df_work['RAIN'] * df_work['T.MAX']
```

A criação das features são importantes porque incorporam conhecimento prévio sobre meteorologia e padrões temporais nos dados, facilitando o aprendizado do modelo. Ao invés de depender exclusivamente do dataset, o modelo recebe informações que mostram como essas variáveis realmente se comportam na natureza. Isso resulta em previsões mais precisas e um modelo que generaliza melhor para períodos não vistos durante o treinamento.

6. Análise Explicativa com LIME e SHAP

Alguns modelos de machine learning, principalmente os mais complexos, conseguem ótimos resultados, mas são difíceis de entender. Para resolver isso, existem ferramentas que ajudam a explicar como esses modelos chegam às previsões. Duas bem conhecidas são o LIME e o SHAP.

- LIME: foca em explicar uma previsão específica. Ele analisa o que levou o modelo a dar aquele resultado para um único exemplo, mostrando quais variáveis puxaram a previsão para cima ou para baixo.

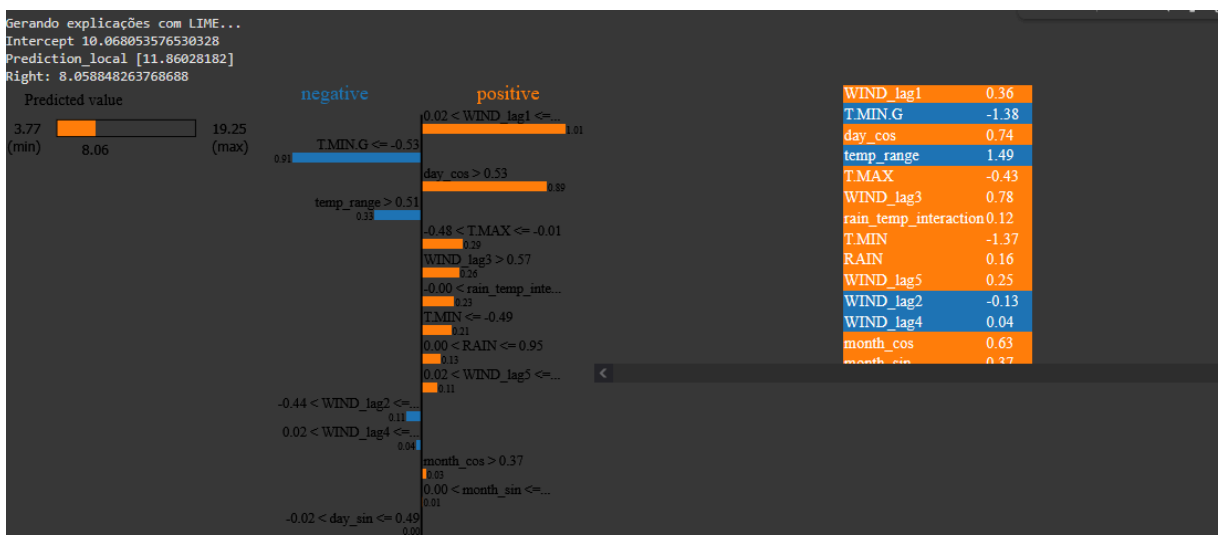
- SHAP: mostra o impacto de cada variável no geral, considerando todas as previsões feitas pelo modelo. Ele ajuda a entender quais são as features mais importantes no comportamento do modelo como um todo, além de também permitir explicações pontuais.

Nos modelos estudados, o que mais se sobressai em resultados é o Random Forest (como consta no gráfico com lag features mostrado anteriormente), dessa forma, irei fazer a análise apenas deste modelo. Vale ressaltar que o estudo dos demais modelos está no colab.

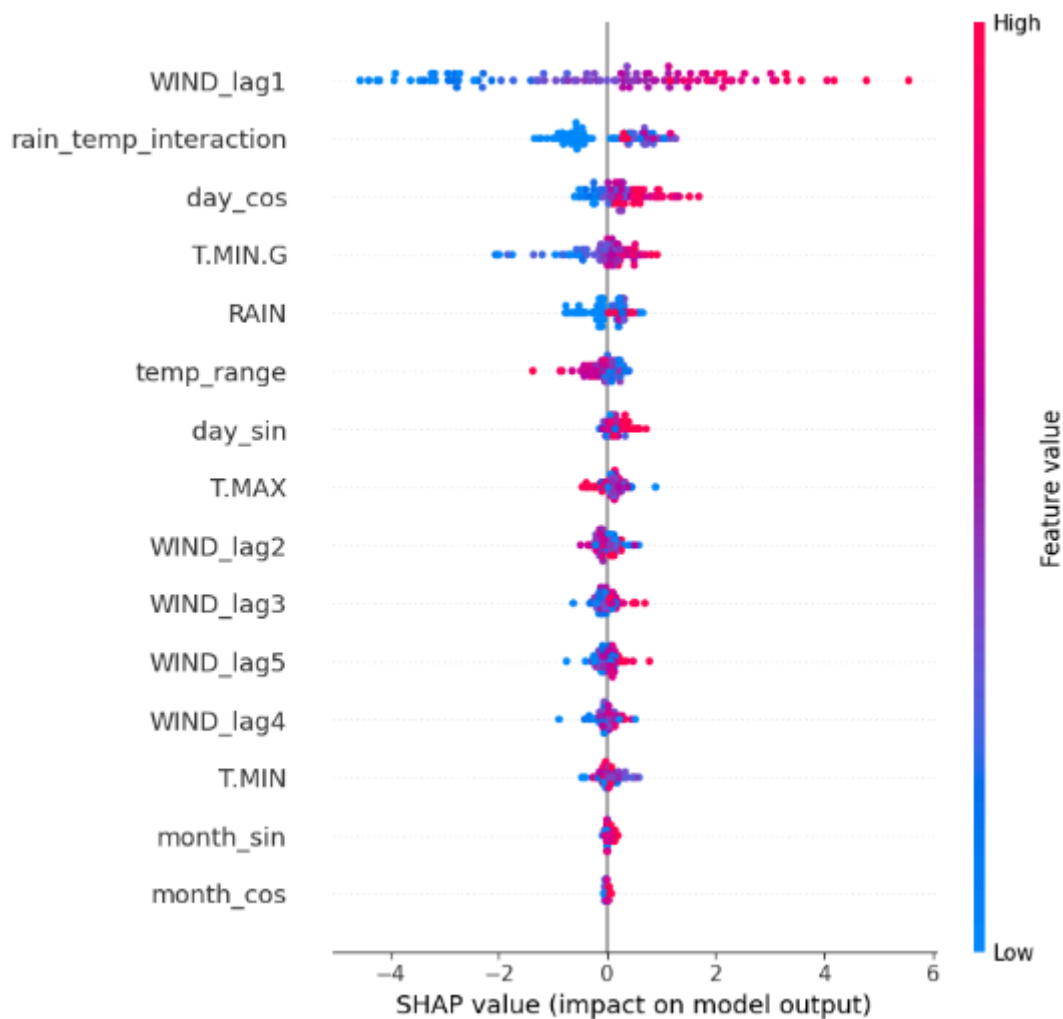
Resumo dos resultados médios por modelo:

	Modelo	MAE médio	RMSE médio	R ² médio
0	random_forest	2.951069	3.751530	0.394715
1	xgboost	2.951615	3.754563	0.393712
3	lasso	2.981016	3.789457	0.381333
2	ridge	2.982183	3.791789	0.380627
4	linear_regression	2.984528	3.795000	0.379672

No gráfico do LIME, conseguimos ver por que o modelo previu um valor em torno de 11,86 para a velocidade do vento em um caso específico, enquanto o valor real era 8,05. A variável que mais puxou a previsão para cima foi a WIND_lag1 — ou seja, a velocidade do vento um dia antes teve grande influência. O day_cos (que representa o horário do dia) também teve impacto positivo. Já variáveis como TMIN.G, TMAX e WIND_lag2 atuaram no sentido oposto, ajudando a reduzir a previsão.



No gráfico do SHAP, temos uma visão geral de todas as previsões. A WIND_lag1 aparece como a feature mais importante, com bastante influência no valor final — quanto maior ela é, mais alta tende a ser a previsão. Outras lags como WIND_lag2, lag3, lag4 e lag5 também aparecem entre as variáveis mais relevantes, o que mostra que o modelo realmente aprendeu com o histórico da velocidade do vento. Além disso, variáveis como rain_temp_interaction, day_cos e temp_range também tiveram papel importante nas decisões do modelo.



Analisando os dados mostrados pelos gráficos, fica ainda mais claro o quanto as lag features foram importantes no projeto. Essas variáveis, que representam valores passados da própria velocidade do vento, tiveram grande influência tanto nas explicações locais (LIME) quanto nas globais (SHAP), aparecendo de forma consistente entre as mais relevantes nas previsões.

A `WIND_lag1` — que corresponde à velocidade do vento no dia anterior — foi a que mais se destacou, mostrando que o modelo realmente se apoia no histórico recente para prever o que vem pela frente. Outras features como `lag2`, `lag3`, `lag4` e `lag5` também tiveram um papel importante, o que reforça que não é só o dia anterior que importa, mas sim um pequeno histórico de dias passados.

Esse comportamento faz sentido com o que já se espera do problema: a velocidade do vento costuma seguir um padrão de continuidade, ou seja, o que aconteceu nas horas ou dias anteriores influencia diretamente o que vem depois. Por isso, incluir essas lag features foi essencial para que o modelo conseguisse captar melhor esses padrões temporais e, assim, gerar previsões mais precisas.

7. Aplicação em problemas práticos

Com a busca cada vez maior por soluções que unem sustentabilidade, eficiência e tecnologia, foi desenvolvido um protótipo de modelo inteligente para prever a velocidade do vento usando técnicas de Machine Learning. A ideia é simples: aproveitar dados meteorológicos e usar o histórico recente (com as chamadas lag features) para fazer previsões mais precisas.

Além de prever, nosso modelo também mostra por que chegou a determinada resposta, usando ferramentas como LIME e SHAP. Isso ajuda a entender quais variáveis mais influenciaram cada previsão, deixando tudo mais claro e confiável.

Essa solução pode ser útil em várias áreas, como:

- Energia eólica, para prever quanto será gerado e planejar o uso das turbinas;
- Agronegócio, em atividades que dependem das condições do vento, como a pulverização;
- Startups de tecnologia climática, que queiram integrar esse tipo de previsão em seus produtos.

É um modelo versátil, explicável e pronto para ser usado ou adaptado conforme a necessidade.

Link para o colab(final): [🔗 ModeloPrediçãoVentoFinal.ipynb](#)

Link para o colab: [🔗 ModeloPrediçãoVento](#)